

DOCUMENTO TECNICO: SISTEMA DE SIGILO



Documento técnico

Sistema de sigilo con IA multicapa: A*, Behavior Tree y sistema multiagente

Enlace al proyecto

GitHub: https://github.com/Egoitz29/Stealth_AI_AStar_BT.git

1. Introducción

En este proyecto se ha desarrollado un sistema de sigilo con inteligencia artificial multicapa en Unity. El objetivo principal era crear un escenario donde dos enemigos patrullan por un mapa, detectan al jugador, reaccionan a su presencia y se comunican entre ellos cuando uno de los guardias entra en estado de alerta.

El sistema no utiliza NavMesh. En su lugar, se ha implementado un grid propio con pathfinding A*. Cada celda del grid puede ser navegable, obstáculo o zona oscura. Las zonas oscuras tienen un coste mayor para los enemigos y también hacen que el jugador sea más difícil de detectar.

La inteligencia artificial de los enemigos está organizada mediante un Behavior Tree, apoyado por un Blackboard que almacena información importante como el estado actual del enemigo, la última posición conocida del jugador, el nivel de sospecha y si el jugador ha sido detectado.

2. Objetivos del sistema

Los objetivos principales del proyecto son crear un grid propio sin usar NavMesh, implementar pathfinding A* sobre ese grid, diferenciar entre casillas navegables, obstáculos y zonas oscuras, hacer que dos enemigos patrullen usando A*, implementar detección del jugador, permitir que el jugador pueda evitar ser detectado en zonas oscuras, crear un sistema de alerta entre guardias y organizar la IA mediante Behavior Tree y Blackboard.

También se ha añadido una visualización del estado de los enemigos y una barra de sospecha para que el funcionamiento del sistema sea más fácil de entender durante la demostración.

3. Implementaciones realizadas

3.1 Grid propio

Se ha creado un sistema de grid mediante los scripts GridManager.cs y GridNode.cs.

Cada celda del grid almacena su posición en el mundo, sus coordenadas dentro del grid, si es navegable o no, si pertenece a una zona oscura, su coste de movimiento y los valores necesarios para el algoritmo A*.

El grid detecta obstáculos y zonas oscuras usando capas de Unity. Para ello se han utilizado las layers Obstacle y DarkZone. Las casillas se visualizan mediante Gizmos en la escena: las casillas verdes representan zonas navegables, las rojas representan obstáculos y las moradas o azuladas representan zonas oscuras.

Esta visualización permite comprobar rápidamente si el mapa está siendo interpretado correctamente por el sistema.

3.2 Casillas con diferentes costes

El sistema diferencia entre casillas normales y casillas oscuras.

Las casillas normales tienen un coste bajo de movimiento, mientras que las casillas oscuras tienen un coste mayor. Esto significa que el enemigo puede atravesar una zona oscura si es necesario, pero el algoritmo A* intentará evitarla cuando exista una ruta alternativa razonable.

De esta manera, las zonas oscuras cumplen dos funciones dentro del sistema: afectan al pathfinding de los enemigos y también ayudan al jugador a evitar ser detectado.

3.3 Pathfinding A*

El movimiento de los enemigos se realiza mediante un algoritmo A* propio implementado en el script AStarPathfinder.cs.

El algoritmo convierte la posición inicial y la posición final en nodos del grid. Después revisa los nodos vecinos, calcula el coste de movimiento, evita las casillas marcadas como obstáculos y tiene en cuenta el coste extra de las zonas oscuras.

Cuando encuentra una ruta válida, reconstruye el camino final usando los nodos padre. El enemigo no se mueve directamente hacia el objetivo, sino que sigue una lista de nodos generada por el sistema A*.

Gracias a esto, los guardias pueden esquivar obstáculos y moverse por el escenario sin utilizar NavMesh.

3.4 Movimiento del jugador

El jugador se mueve libremente por el escenario mediante el script `PlayerMovement.cs`. El movimiento se realiza con teclado, usando WASD o las flechas.

Además, el jugador tiene el script `PlayerStealth.cs`, que comprueba constantemente en qué tipo de casilla se encuentra. Si el jugador está sobre una casilla oscura, el sistema guarda que se encuentra en una zona de oscuridad.

Esta información es utilizada por los guardias para reducir la detección del jugador cuando está dentro de una zona oscura.

3.5 Behavior Tree

La IA de cada guardia está organizada mediante un Behavior Tree. Este árbol decide qué debe hacer el enemigo en cada momento según la situación.

La estructura general del árbol es la siguiente:

Selector principal

Si ve al jugador, lo persigue o lo ataca.

Si está alertado, investiga la última posición conocida.

Si está en sospecha, mira hacia la zona sospechosa.

Si no ocurre nada, patrulla.

El Behavior Tree permite que el enemigo tenga prioridades claras. La prioridad más alta es detectar al jugador. Después está el estado de alerta, luego la sospecha y finalmente la patrulla.

3.6 Blackboard

Cada enemigo tiene un `EnemyBlackboard.cs`. El Blackboard almacena la información importante que necesita la IA para tomar decisiones.

Guarda la referencia al jugador, si el enemigo puede ver al jugador, si el jugador está en una zona oscura, la última posición conocida del jugador, el estado actual del enemigo y la cantidad de sospecha acumulada.

Los estados principales del enemigo son Patrol, Suspicious, Alert y Search.

Separar la información en un Blackboard permite que el Behavior Tree consulte los datos necesarios sin mezclar toda la lógica en una sola parte del código.

3.7 Estados de la IA

Los guardias tienen varios estados visibles durante la demo.

En estado de patrulla, el enemigo sigue sus puntos de patrulla usando A*. Visualmente aparece como PATRULLA y usa color cian.

En estado de sospecha, el enemigo ha empezado a ver al jugador pero todavía no lo ha confirmado. Visualmente aparece como SOSPECHA y usa color amarillo. En este estado también se muestra la barra de sospecha.

En estado de alerta, el enemigo ha detectado al jugador. Visualmente aparece como ALERTA y usa color rojo. En este estado persigue al jugador y avisa al otro guardia.

En estado de búsqueda, el enemigo se dirige a la última posición conocida del jugador y busca durante unos segundos. Visualmente aparece como BUSCANDO y usa color naranja. Si no vuelve a encontrar al jugador, regresa a patrullar.

3.8 Detección del jugador

La detección del jugador tiene en cuenta varios factores: la distancia al jugador, el ángulo de visión del guardia, los obstáculos que puedan bloquear la visión, si el jugador está en una zona oscura y el tiempo durante el cual el jugador permanece visible.

Cuando el jugador está en una zona normal, el guardia lo detecta rápido. En cambio, cuando el jugador está en una zona oscura, el rango de detección se reduce y el tiempo necesario para detectarlo aumenta.

Esto permite que el jugador pueda usar las zonas oscuras para evitar ser detectado. Si entra en una zona oscura y sale del campo de visión antes de que la barra de sospecha se llene, el guardia no llega a entrar en alerta.

3.9 Sistema multiagente

El sistema multiagente se ha implementado mediante GuardAlertSystem.cs.

Cuando un guardia detecta al jugador, entra en estado de alerta, guarda la última posición conocida del jugador y avisa al sistema global de alerta. Este sistema comunica la información al resto de guardias.

El segundo guardia recibe la alerta, deja de patrullar y se dirige a investigar la última posición conocida del jugador. Esto hace que los enemigos no actúen de forma totalmente aislada, sino que compartan información relevante entre ellos.

3.10 Barra de sospecha

Para hacer más claro el funcionamiento del sigilo, se ha añadido una barra de sospecha encima de cada guardia.

La barra aparece cuando el guardia empieza a sospechar del jugador. En zona normal, la barra sube rápido. En zona oscura, la barra sube más despacio. Cuando el jugador es detectado, la barra se llena completamente y pasa a color rojo.

Esta barra ayuda a demostrar visualmente que las zonas oscuras afectan realmente al sistema de detección.

4. Problemas encontrados

Problema 1: los guardias se quedaban parados al llegar a un punto

Al principio, los guardias llegaban cerca de un punto de patrulla, pero no avanzaban al siguiente. Esto ocurría porque el A* lleva al enemigo al centro de una casilla, mientras que el punto de patrulla puede no estar exactamente en ese centro.

Para solucionarlo, se aumentó la distancia de llegada del enemigo y también se modificó la lógica para considerar que el guardia ha llegado cuando termina el path, aunque no esté exactamente encima del punto de patrulla.

Problema 2: los dos guardias iban al mismo punto

Al usar los mismos puntos de patrulla, los dos guardias empezaban en el mismo índice y se dirigían al mismo objetivo.

Para solucionarlo, se añadió una variable llamada Starting Patrol Index. Gracias a esto, cada guardia puede empezar en un punto diferente de la ruta. Por ejemplo, Guard_01 puede empezar en el índice 0 y Guard_02 en el índice 2.

Problema 3: la detección en oscuridad no se apreciaba bien

Aunque el sistema reducía la detección en las zonas oscuras, al principio no se veía claramente durante la demo.

Para solucionarlo, se añadió una barra de sospecha encima de cada guardia. Así se puede comprobar que fuera de la oscuridad la barra sube rápido, mientras que dentro de la oscuridad sube más despacio.

Problema 4: era difícil entender el estado de cada enemigo

Al principio solo se veía el movimiento de los guardias, pero no quedaba claro qué estaba haciendo internamente la IA.

Para solucionarlo, se añadió texto encima de cada guardia indicando su estado actual: PATRULLA, SOSPECHA, ALERTA o BUSCANDO.

Esto facilita entender el funcionamiento del Behavior Tree y del Blackboard durante la demostración.

5. Soluciones aplicadas

Las principales soluciones aplicadas fueron crear un grid propio para controlar completamente la navegación, usar A* en lugar de NavMesh, añadir costes distintos según el tipo de celda, separar la lógica de decisión mediante Behavior Tree, almacenar la información de cada enemigo en un Blackboard, crear un sistema global de alerta entre guardias, añadir visualización de estados y sospecha, y ajustar las distancias de llegada para evitar bloqueos en el movimiento.

6. Resultado final

El resultado final es un prototipo funcional de sigilo con IA multicapa.

El sistema permite mover al jugador libremente, hacer que dos guardias patrullen usando A*, evitar obstáculos mediante pathfinding propio, usar zonas oscuras como ventaja para el jugador, detectar al jugador según distancia, ángulo, obstáculos y oscuridad, mostrar una barra de sospecha, cambiar estados mediante Behavior Tree, guardar información en Blackboard, alertar a otro guardia cuando uno detecta al jugador, investigar la última posición conocida y volver a patrullar si no se encuentra al jugador.

7. Conclusión

Este proyecto cumple los requisitos principales de la práctica, ya que combina un sistema de pathfinding A* propio, un Behavior Tree para la toma de decisiones, un

Blackboard para almacenar información de la IA y un sistema multiagente para compartir alertas entre guardias.

Además, las zonas oscuras añaden una capa de sigilo, ya que permiten al jugador reducir su visibilidad y evitar ser detectado si se mueve con cuidado.

El sistema es ampliable. En el futuro podrían añadirse mejoras como pathfinding asíncrono, costes dinámicos de luz y oscuridad en tiempo real, más niveles de alerta, sonidos para distraer enemigos o un sistema de vida real para el jugador.